

Contents

1 Photocatalysis stage-1 gap filter	1
1.1 What runs, on what	1
1.2 Failure probability (the ensembling, made quantitative)	1
1.3 Where the results land	1
1.4 Configuration	2
1.5 Deployment env — IMPORTANT (python version!)	2
1.6 Files	2

1 Photocatalysis stage-1 gap filter

Two-stage experimental-target band-gap filter for photocatalyst candidates. Stage 1 (this module, automated): after the catalysis chain of ANY reaction path completes, an ensemble of pretrained gap models scores the bulks behind the best-eta slabs and tags those slabs in the SQL DB with a gap record and a **failure probability** against the requested window. Stage 2 (manual, by design): HSE on the shortlist — the pipeline never launches it.

1.1 What runs, on what

- Selection: the `n_slabs` (default 10) slabs with the lowest best-eta for the submission's (reaction, reaction_path), from DBSurfaceMLAdsorbate.
- Prediction target: the **parent bulk structure** of each slab (lowest- energy DBStructureVersion on the bulk_relax method). Bulk-trained gap models on a slab-with-vacuum would be off-distribution garbage — the slab carries the tag, the bulk carries the physics. One prediction per unique bulk, shared by all its slabs.
- Ensemble (each backend optional at runtime; whatever imports, runs):

backend	training target	fidelity role
alignn_mbj	JARVIS TBmBJ gaps	experimental-target (primary)
modnet_expt	experimental gaps	experimental-target
alignn_opt	JARVIS OPT (PBE)	consistency check only
megnet_pbe	MP PBE gaps	consistency check only

1.2 Failure probability (the ensembling, made quantitative)

$P(\text{true gap outside } [\text{gap}, \text{gap_max}])$ under $\text{Normal}(\text{gap_mean}, \text{sigma_eff})$:

$$\text{sigma_eff}^2 = \text{sigma_model}^2 + (\text{spread}/2)^2 + (0.25 \text{ eV} * \text{n_flags})^2$$

- gap_mean, spread: over the experimental-target members only.
- sigma_model (default 0.5 eV): the honest chain error vs experiment (ALIGNN-vs-MBJ ~0.31 eV stacked on MBJ-vs-exp ~0.3-0.5 eV).
- Suspicion flags widening sigma: `d10_cu_ag_mbj_unreliable` (mBJ fails Cu(I)/Ag d10 compounds — Cu2O ~1 eV low), `fidelity_inversion` (a PBE member ABOVE the experimental-target mean is physically backwards).
- Informational flags: `single_expt_target_model`, `predicted_metal`, `no_expt_target_model` (then `p_fail` = 1).

1.3 Where the results land

- `DBSurface.attributes["photocat"][reaction][reaction_path]` = {eta, miller_index, gaps per model, gap_mean, spread, sigma_eff, p_fail, flags, window, per-model errors}. Written with

- an atomic nested jsonb_set (db.utils.update_jsonb_path), so parallel reactions tagging the same slab never clobber each other.
- DBStructureVersion.attributes["photocat_gap"] on the bulk (once).
 - Export: export_all.py carries the tag on every surface record; export_all.py --all --photocat-only dumps ONLY the tagged slabs with their gap records, the etas that selected them, and the bulk structures (always embedded in that mode; --with-structures adds the slab dicts).

1.4 Configuration

```
# input.yaml
photocat:
  enabled: true
  gap: 1.8          # eV; window lower edge
  gap_max: 3.5      # eV; upper edge; 'none' disables it
  n_slabs: 10
  models: [alignn_mbj, alignn_opt, megnet_pbe, modnet_expt]
  sigma_model: 0.5

# config.yaml – the stage skips loudly when this is absent
codes:
  photocat:
    code_string: photocat@<computer>
    job_script: {nodes: 1, ntasks: 1, cpus: 4, time: 3600, exclusive: false}
```

1.5 Deployment env — IMPORTANT (python version!)

ALIGNN's pretrained checkpoints instantiate the dgl-based architecture, and **dgl has no python-3.14 wheels (cp312 is the ceiling)** — verified: on a 3.14 env pip silently falls back to a broken 0.x sdist. The photocat code therefore CANNOT point at the mace_3.14 venv; give it its own small env:

```
python3.12 -m venv photocat_env
pip install alignn dgl -f https://data.dgl.ai/wheels/repo.html # cpu dgl
# optional extra ensemble members:
pip install megnet          # brings tensorflow
pip install modnet          # + set MODNET_EXPT_GAP_MODEL=<saved model>
```

CPU is fine — inference on ~10 structures is seconds. A first run downloads the figshare checkpoints to the node's cache; on air-gapped compute nodes, pre-warm the cache from a login node.

1.6 Files

engine/runner codes/files/photocat_gap.py (pure ensemble math unit-tested in tests/test_photocat_gap.py incl. a mock-backend end-to-end runner pass) · plugin codes/photocat/{calculation,parser,workchain}.py · orchestratorworkchains/photocat.py · gating main.py (step_status["photocat"][reaction][path], needs adsorbates Done) · DB helper db.utils.update_jsonb_path.